

Automatic Water Tank Refill & Level Indicator System

Comprehensive Engineering Technical Documentation & Implementation Guide

1. Project Overview

Water wastage due to tank overflow and pump damage caused by dry running are common household and industrial challenges. This project provides an efficient, plug-and-play solution using an **Arduino Uno** as the central microcontroller. By continuously monitoring three critical structural thresholds (**10%**, **50%**, and **90%**), the system safely automates the refilling process and provides local visual and acoustic telemetry without requiring human intervention.

2. System Logic & Operational Workflow

The system samples data from three sensors precisely placed at distinct height intervals inside the water reservoir. The automation sequence operates on a strict feedback loop sequence:

Level 1: Critical Low ($\leq 10\%$)

- **Visual Indication:** Red LED illuminates to show critical depletion.
- **Audio Indication:** Piezo buzzer emits a continuous alarm tone.
- **Control Action:** Output relay energizes, turning **ON** the water pump to initiate refilling.

Level 2: Medium State (50%)

- **Visual Indication:** Blue LED illuminates; the Red LED goes dark.
- **Audio Indication:** Piezo buzzer ceases all alarm signals.
- **Control Action:** The water pump remains **ON**. Latching firmware logic holds the relay state until the high-level boundary is reached.

Level 3: Maximum Full ($\geq 90\%$)

- **Visual Indication:** Green LED illuminates; the Blue LED goes dark.
- **Audio Indication:** Piezo buzzer sounds a distinct 4-burst notification beep sequence and then silences.
- **Control Action:** Output relay instantly de-energizes, turning **OFF** the water pump to prevent overfilling.

3. Hardware Components & Pin Configurations

The prototype circuit architecture maps all essential peripheral modules to the Arduino Uno processing core as detailed below:

Component Description	Qty	Functional Assignment	Arduino Connection Pin
Arduino Uno R3	1	Central Processing Unit / Main Controller	Core Board Reference
Soil Moisture Probe Modules	3	Conductivity Level Sensors (10% , 50% , 90%)	A0, A1, A2 (Digital Mode)
2-Channel Relay Module	1	High-Voltage Power Isolation Switch (Channel 1)	Pin 6
Red LED Indicator	1	Visual Telemetry for 10% Low Level	Pin 2
Blue LED Indicator	1	Visual Telemetry for 50% Mid Level	Pin 3
Green LED Indicator	1	Visual Telemetry for 90% Full Level	Pin 4
Active Piezo Buzzer	1	Acoustic Warning / Notification Unit	Pin 5
Resistors (220Ω)	3	Current Limiting Protection for Indicators	Inline with LED Anodes

4. Schematic & Wiring Instructions

- **Power Supply Rail:** Route the Arduino 5V and GND pins directly onto the primary power rails of the structural breadboard. Divert all peripheral power leads to these standard reference rails.
- **Sensor Configuration:** Link the active Digital Output (DO) channels of the Low, Medium, and High level threshold probes directly to Arduino input channels A0, A1, and A2.
- **Output Interfaces:** Place a 220Ω resistor in series with the long positive legs (Anodes) of the Red, Blue, and Green LEDs, tying their control lines to digital pins 2, 3, and 4. Route the active buzzer line to pin 5. Ground all common cathode nodes.
- **High-Voltage Isolation:** Split the main alternate current Phase line supplying the external water pump motor. Land the input line directly into the **COM (Common)** terminal block and secure the pump return onto the **NO (Normally Open)** terminal.

5. Firmware Source Code

The structured microcontroller code implements reliable sensor debouncing check loops alongside active-low state driving for the electrical relays:

```

// --- Pin Configurations ---
const int sensor10 = A0;    // Bottom Sensor
const int sensor50 = A1;    // Middle Sensor
const int sensor90 = A2;    // Top Sensor

const int redLed = 2;       // 10% Indicator
const int blueLed = 3;      // 50% Indicator
const int greenLed = 4;     // 90% Indicator

const int buzzer = 5;       // Alert Buzzer
const int relayPump = 6;    // 2-Channel Relay (Channel 1)

// --- System Variables ---
bool pumpState = false;
bool topBuzzerDone = false;

void setup() {
    pinMode(sensor10, INPUT);
    pinMode(sensor50, INPUT);
    pinMode(sensor90, INPUT);

    pinMode(redLed, OUTPUT);
    pinMode(blueLed, OUTPUT);
    pinMode(greenLed, OUTPUT);
    pinMode(buzzer, OUTPUT);
    pinMode(relayPump, OUTPUT);

    // Default States (Assuming Active LOW Relay)
    digitalWrite(relayPump, HIGH);
    digitalWrite(redLed, LOW);
    digitalWrite(blueLed, LOW);
    digitalWrite(greenLed, LOW);
    digitalWrite(buzzer, LOW);
}

void loop() {
    bool isWet10 = (digitalRead(sensor10) == LOW);
    bool isWet50 = (digitalRead(sensor50) == LOW);
    bool isWet90 = (digitalRead(sensor90) == LOW);

    // Level 1: Below 10%
    if (!isWet10) {
        digitalWrite(redLed, HIGH);
        digitalWrite(blueLed, LOW);
        digitalWrite(greenLed, LOW);
        digitalWrite(buzzer, HIGH);
        pumpState = true;
        digitalWrite(relayPump, LOW); // Turn ON Pump
        topBuzzerDone = false;
    }

    // Level 2: 50% Filled
    else if (isWet10 && !isWet90) {
        digitalWrite(redLed, LOW);
        digitalWrite(blueLed, HIGH);
        digitalWrite(greenLed, LOW);
        digitalWrite(buzzer, LOW);

        if (pumpState) digitalWrite(relayPump, LOW);
        else digitalWrite(relayPump, HIGH);
    }
}

```

```

}

// Level 3: 90% or Above
else if (isWet90) {
    digitalWrite(redLed, LOW);
    digitalWrite(blueLed, LOW);
    digitalWrite(greenLed, HIGH);
    digitalWrite(buzzer, LOW);

    pumpState = false;
    digitalWrite(relayPump, HIGH); // Turn OFF Pump

    if (!topBuzzerDone) {
        for (int i = 0; i < 4; i++) {
            digitalWrite(buzzer, HIGH);
            delay(200);
            digitalWrite(buzzer, LOW);
            delay(200);
        }
        topBuzzerDone = true;
    }
}
delay(500);
}

```

6. Advantages & Strategic Benefits

- **Resource Conservation:** Eradicates manual monitoring errors, saving water resources and eliminating energy drain.
- **Equipment Protection:** Complete integration of dry-run safety systems ensures the pump motor is safeguarded against stator core overheating.
- **Scalable Implementation:** Built entirely out of low-cost prototyping modules that are easily serviceable.